

Gestión de Ficheros

Profesor: Juan Montes de Oca

Alumnos: Jaume Adrover Fernandez y Joan Balaguer Llagostera

Assignatura: Programació I



1. Introducción

En esta práctica se nos pedía llevar a cabo un programa que, a partir del nombre de un fichero dado por el usuario, tratase el contenido de este, con el fin de que el mismo usuario recibiese cierta información relevante sobre el fichero introducido (número de caracteres, palabras y líneas del fichero). Además, el mismo usuario, tendrá diversas opciones de tratamiento del fichero para que pueda realizar diversas tareas en relación al contenido del mismo. Estas son:

- 1- Mostrar la letra más repetida y su número de apariciones.
- 2- Mostrar el número de apariciones de cada carácter.
- 3- Mostrar la palabra más repetida y su número de apariciones.
- 4- Buscar una palabra del fichero.
- 5- Buscar un texto del fichero.
- 6- Buscar las palabras repetidas
- 7- Codificación del fichero tratado.
- 8- Decodificación del fichero tratado

2. Diseño descendente del programa

La primera tarea que debemos realizar para llevar a cabo el programa, es la lectura del nombre del fichero que nos va a introducir el usuario. Para ello, hemos declarado una variable de tipo String (llamada nombreFichero) para almacenar en ella el contenido de la línea que nos va a introducir (la cual, vamos a leer con el método de la clase LT, `readLine()`). Además, previamente le habremos informado de que debe introducir el nombre del fichero que desea analizar, mediante un `System.out.print`.

Una vez que ya tenemos el nombre del fichero que el usuario quiere analizar, almacenado en la variable String, ya podemos focalizarnos en el análisis del fichero introducido. Para mostrar el número de caracteres, hemos creado un método llamado `GetNumeroCaracteres()`. Este, mediante un objeto `BufferedReader`, lleva a cabo el tratamiento del fichero introducido a nivel de caracteres para acabar retornando el número de caracteres del fichero del usuario. Para determinar el número de Palabras del fichero, hemos declarado un objeto `PalabraFicherosIn`, llamado `fichero1`, el cual hemos enlazado con el nombre del fichero que nos ha introducido el usuario. Una vez enlazado, mediante el método de objeto `GetNumeroPalabras()` de la clase `PalabraFicherosIn`, llevamos a término el recuento de palabras del fichero. Por último, para contar el número de líneas del fichero, seguiremos la misma estrategia que para contabilizar el número de palabras pero en este

caso con la clase `LineasFicherosIn` y su correspondiente método `GetNumeroLineas()`. Estos 3 datos se van a mostrar al usuario mediante 3 `System.out.println()`.

Como el límite de palabras del fichero introducido no puede ser superior a 500, establecemos una condición inicial para poder tratarlo: si el número de palabras del fichero (obtenido mediante el método de la clase `PalabrasFicherosIn`, `GetNumeroPalabras`) es mayor que 500, vamos a informarle al usuario que su fichero no puede ser tratado.

Una vez que ya hemos mostrado los datos iniciales al usuario, este, debe elegir entre 7 opciones de un menú que hemos creado a partir de un bucle switch, el cual, en función del número que nos introduzca el usuario mediante el teclado, llevará a cabo uno u otro método del menú. Además mediante el método `borrarPantalla()`, cada vez que el usuario introduce un número para elegir una opción del menú, se llevará a cabo un `System.out.print` de múltiples saltos de línea, con el cual, las opciones que previamente haya utilizado el usuario, no se verán en pantalla. Una vez explicado cómo funciona el menú, vamos a explicar cada uno de los métodos implícitos en él:

2.1 Método `ObtenerLetraMasRepetida()`

Si el usuario introduce el número 1, se va a llevar a cabo el método `ObtenerLetraMasRepetida()`. Este, mediante la declaración de un objeto `FrecuenciaCaracteres` (llamado frecuencia), un objeto `BufferedReader` (al que le hemos enlazado el fichero introducido por el usuario) y una variable entera llamada código, lleva a cabo un bucle de tratamiento donde, cada vez que se lee un carácter del fichero con el método `read()` de la clase `BufferedReader`, actualiza la frecuencia del carácter leído en el objeto frecuencia mediante el método de objeto de la clase `FrecuenciaCaracteres`, `actualizacion()`. Posteriormente cerramos el fichero tratado. Finalmente, llevamos a cabo un `System.out.print`, informando al usuario de cuál es el carácter del objeto frecuencia que tiene más apariciones. Esto, lo llevamos a cabo mediante el método que hemos creado en la clase `FrecuenciaCaracteres` llamado `LetraMasRepetida()`, el cual nos retorna la componente del array caracteres que tenga el mayor número de apariciones. También informamos al usuario del número de apariciones de dicho carácter mediante el método `obtenerFrecuencia()` de la clase `FrecuenciaCaracteres`.

2.2 Método ObtenerFrecuenciaChar()

Si el usuario introduce el número 2, el programa ejecutará el método `ObtenerFrecuenciaChar()`, el cual lleva a cabo la visualización de la frecuencia de todos los caracteres del fichero. Para ello, declaramos un objeto frecuencia perteneciente a la clase `FrecuenciaCaracteres`, además de un objeto `BufferedReader` (que enlazaremos con el fichero que nos han introducido el usuario) y una variable entera llamada código para poder leer los códigos de carácter del fichero. Luego llevamos a cabo un bucle de tratamiento del fichero, donde a medida que vamos leyendo cada código de carácter del fichero, vamos actualizando el número de apariciones de dicho carácter en el objeto frecuencia mediante el método `actualización()` de la clase `FrecuenciaCaracteres`. Una vez tratados todos los códigos de caracteres del fichero, visualizaremos todas la frecuencias del array frecuencias mediante el método `toString()`, que nos proporciona la clase `FrecuenciaCaracteres`. Finalmente cerramos el fichero tratado con el método `close()` de la clase `BufferedReader`.

2.3 Método ObtenerPalabrasMasRepetidas()

En el caso que el usuario nos introduzca el número 3, el programa iniciará el método `ObtenerPalabrasMásRepetidas()`. Este método, lleva a cabo el tratamiento de todas las palabras del fichero, para acabar visualizando la que más veces haya aparecido y en caso de haber empate, mostraría todas las que estuvieran empatadas. Esta tarea la llevaremos a cabo con la declaración de un objeto `FrecuenciaPalabras` (frecuencia), de un objeto `Palabra` (pal) y un objeto `PalabraFicherosIn` (fichero), el cual enlazaremos con el fichero introducido. Luego llevaremos a cabo un bucle de tratamiento para contabilizar el número de apariciones de cada palabra que vamos leyendo (con el método `lectura()` de la clase `PalabraFicherosIn`) del fichero en el objeto frecuencia, mediante el método de la clase `FrecuenciaPalabras`, `actualización()`. Una vez tenemos la frecuencia de aparición de cada palabra visualizaremos la palabra que tenga el mayor número de apariciones en el fichero, con los métodos `getPalabra()`, `IndicePalabraMasRepetida()` y `getFrecuencia()`, todos de la clase `FrecuenciaPalabras`. Para tratar el caso del empate, realizamos un último bucle de tratamiento donde comparamos la palabra con mayor frecuencia con las demás. En caso de haber empate de frecuencias, las palabras que estuviésemos tratando en ese preciso instante del bucle, también serán visualizadas juntamente a la palabra de mayor frecuencia que habíamos visualizado previamente. Por lo tanto, el caso del empate quedaría resuelto. Por último cerramos el fichero que hemos tratado con el método de la clase `PalabraFicherosIn` `cerrarEnlaceFichero()`.

2.4 Método BuscarPalabra()

En el caso que el usuario nos introduzca el número 4, se ejecutará el método BuscarPalabra, el cual pregunta al usuario que introduzca una palabra que quiera buscar y como respuesta, el programa devuelve la fila y la columna donde se encuentra dicha palabra. Para ello, declaramos un objeto Palabra (palabra) sin inicializar y otro objeto Palabra (palabraBuscada) inicializado con el constructor vacío de la clase Palabra, además de un objeto PalabraFicherosIn para leer el contenido del fichero. Primero de todo llevamos a cabo la lectura de la palabra que nos han introducido. Esto lo hacemos mediante el método lectura() de la clase palabra y introducimos el contenido leído dentro del objeto palabraBuscada. Luego, establecemos el enlace del objeto PalabraFicherosIn con el fichero que vamos a tratar. Por último, llevamos a cabo un bucle de tratamiento para leer todas las palabras del fichero y establecer una condición para que la palabra tratada en cada instante, pueda ser mostrada al usuario con su respectiva columna y línea. Esta es que si la palabra que estamos tratando en el bucle de tratamiento, es igual a la palabraBuscada, entonces dicha palabra tratada será mostrada al usuario. Esta comparación, la llevan a cabo mediante el método de la clase Palabra iguales(), que nos dice si dos palabras son iguales o no. Por último, cerramos el fichero con el método cerrarEnlaceFichero() de la clase PalabraFicherosIn.

2.5 Buscar texto()

En este quinto método nuestra funcionalidad consiste en introducir un texto por teclado y obtener su posición en el fichero. Este método vendría a ser un Linea.Contienelínea(), que es el método de objeto que hemos utilizado en el subprograma BuscarTexto(). El primer paso consiste en pedir al usuario que introduzca un texto a ser buscado que será leído mediante un método línea.lectura(). Después elaboramos un bucle de tratamiento que se llevará a término mientras haya líneas en el fichero. Primero vamos leyendo línea a línea y si esta contiene el texto introducido haremos un system.out.print con su respectiva posición en el fichero. A medida que esto ocurre vamos incrementando el número de líneas de los ficheros.

2.6 Buscar palabras repetidas()

En esta función del menú nuestro objetivo es contabilizar las parejas de palabras iguales que están en el fichero de texto, indicando su línea y columna. Nuestro bucle de tratamiento consiste en hacer una lectura mediante un objeto palabra e ir almacenando su contenido en

otro objeto palabra. En medio de este bucle, hacemos una llamada al método de objeto palabra `pal.Iguals(pal2)`, donde comparamos los objetos `palabra1` y `palabra2`. Si la condición se cumple visualizamos la pareja de palabras y sus respectivas líneas y columnas. Esto es debido a un método aparte nombrado como `visualización(pal1,pal2)`.

2.7 CodificarFichero(semilla,código,nombrefichero)

Al introducir el número 7 se lleva a cabo el método `CodificarFichero()`, muy importante en nuestro programa. En primer lugar, mostraremos la semilla actual al usuario. Después declaramos los objetos correspondientes de las clases `BufferedReader`, `BufferedWriter` y `Codificación alfabética`. Procedemos a leer el primer carácter para inicializar la variable `código` y entrar en el bucle de tratamiento donde codificaremos cada carácter leído. Todo el contenido codificado se almacenará en un fichero llamado igual que el original, añadiendo `".cod.txt"`. Este método termina con los cierres de enlaces de los ficheros de entrada y salida respectivos.

2.8 DecodificarFichero(semilla,codigo,nombrefichero)

La opción número 8 del usuario consiste en obtener el contenido original de un fichero codificado. Obtenemos un fichero `"x.cod.txt.txt"` y decodificamos íntegramente su texto en otro fichero de salida con el nombre `"x"`. El método `GetFicheroDecod("nombrefichero+txt")` nos permite extraer el nombre del fichero que decodificado, haciendo un bucle de lectura hasta que encontremos el primer punto del nombre. Por ejemplo: `joan.cod.txt.txt-----> joan.txt`. En cuanto al método `DecodificarFichero()` en sí consiste en realizar un bucle de lectura similar al método anterior. A medida que leemos un carácter, escribimos su versión decodificada en otro fichero de salida con el supuesto nombre original. Terminaremos este proceso con los cierres de enlaces de los ficheros.

3. Juego de Pruebas

Este fichero llamado `jj.txt`, será el que usaremos para poner a prueba el funcionamiento de nuestro programa. El contenido de este es el siguiente:

```
jj.txt — Modificado
hola muy buenas a todosa esta es la practica final de programaciòn de joan y yo
nuestro trabajo consiste en analizar un fichero
fichero de de texto texto esta practica es muy
importante a nivel de fichero jaume
jaume jaume jaumb
```

-Lo primero que nos encontramos al ejecutar el programa, és lo siguiente:

```
Introducir nombre del fichero a analizar: |
```

Nosotros queremos analizar el fichero llamado jj.txt, por lo tanto introducimos dicho nombre al programa (cabe recordar que el fichero debe estar contenido en el proyecto NetBeans que estamos utilizando para que funcione correctamente).

```
Introducir nombre del fichero a analizar: jj.txt
El número de caracteres del fichero es de: 233
El número de palabras del fichero es de: 41
El número de líneas del fichero es de: 5

                OPCIONES DE ANÁLISIS
(1) MOSTRAR LA LETRA MÁS REPETIDA Y SU NUMERO DE APARICIONES.
(2) MUESTRA EL NUMERO DE APARICIONES DE CADA CARACTER.
(3) MUESTRA LA PALABRA MÁS REPETIDA Y SU NÚMERO DE APARICIONES.
(4) BUSCA UNA PALABRA.
(5) BUSCA UN TEXTO.
(6) BUSCA LAS PALABRAS REPETIDAS.
(7) CODIFICA FICHERO.
(8) ESTABLECE LA SEMILLA DE DECODIFICACIÓN PARA DECODIFICAR EL FICHERO.
(0) SALIR.

ESCOGER OPCIÓN -->
```

Como vemos, nada más introducir el nombre del fichero, el programa ya nos muestra el número de caracteres palabras y líneas de este. Además, se nos despliega un menú indicando al usuario, que numero debe introducir para llevar a término cada acción.

-Empezaremos introduciendo el número 1, el cual tiene como resultado de su ejecución:

```
La letra más repetida és: -----> frecuencia 39
```

Como vemos, el carácter “espacio en blanco” es el más repetido con 39 apariciones.

-Si introducimos el número 2, el resultado de la ejecución es el siguiente:

```
Frecuencia caracter a: 26.  
Frecuencia caracter b: 3.  
Frecuencia caracter c: 9.  
Frecuencia caracter d: 6.  
Frecuencia caracter e: 24.  
Frecuencia caracter f: 4.  
Frecuencia caracter g: 1.  
Frecuencia caracter h: 4.  
Frecuencia caracter i: 11.  
Frecuencia caracter j: 6.  
Frecuencia caracter k: 0.  
Frecuencia caracter l: 5.  
Frecuencia caracter m: 8.  
Frecuencia caracter n: 11.  
Frecuencia caracter ñ: 0.  
Frecuencia caracter o: 16.  
Frecuencia caracter p: 4.  
Frecuencia caracter q: 0.  
Frecuencia caracter r: 11.  
Frecuencia caracter s: 9.  
Frecuencia caracter t: 14.  
Frecuencia caracter u: 9.  
Frecuencia caracter v: 1.  
Frecuencia caracter w: 0.  
Frecuencia caracter x: 2.  
Frecuencia caracter y: 4.  
Frecuencia caracter z: 1.  
Frecuencia caracter : 39.  
Frecuencia caracter .: 0.  
Frecuencia caracter ,: 0.  
Frecuencia caracter :: 0.  
Frecuencia caracter @: 0.  
Frecuencia caracter ?: 0.  
Frecuencia caracter !: 0.  
Frecuencia caracter ": 0.  
Frecuencia caracter (: 0.  
Frecuencia caracter ): 0.  
Frecuencia caracter <: 0.  
Frecuencia caracter >: 0.
```

Como vemos, se nos muestra el número de apariciones de cada carácter.

-Si queremos realizar la tercera opción vamos a marcar el número 3 y tendrá como resultado la siguiente ejecución:

```
Palabras más repetidas:  
Palabra : de-----> frecuencia 5
```

En este caso, la palabra más repetida del fichero jj.txt es la palabra “de”, que aparece 5 veces.

-Para llevar a cabo la búsqueda de una palabra, marcaremos la opción número 4 la cual nos pedirá que le introduzcamos una palabra al programa. Nosotros hemos elegido la palabra “jaume” y el programa nos devolverá su posición exacta en el fichero. Si hay más de una posición, es que la palabra aparece más de una vez en el fichero, como es el caso.


```
Introducir una palabra que quieras buscar:jaume
La palabra jaume se encuentra en: La línea 4 y en la columna 31
La línea 5 y en la columna 1
La línea 5 y en la columna 7
```

-Si queremos buscar un texto en el fichero solo tenemos que introducir el número 5. El programa nos pedirá que le introduzcamos un texto. Para este ejemplo, hemos elegido el texto “nuestro trabajo”, que está contenido en el fichero en la segunda línea.

```
Introduce texto a ser buscado:nuestro trabajo
El texto: nuestro trabajo se encuentra en la línea 2
```

En caso que el texto introducido no estuviera contenido en el fichero, el programa simplemente no visualizará nada, seguiría con su ejecución normal del menú.

-Si introducimos el número 6, el programa nos mostrará todas la parejas de palabras repetidas que están contenidas en el fichero, indicando la línea y la columna donde se encuentran dichas palabras repetidas. En el caso de este fichero jj.txt:

```
Pareja 1
Palabra: fichero --> línea: 2, columna: 42
Palabra: fichero --> línea: 3, columna: 1

Pareja 2
Palabra: de --> línea: 3, columna: 9
Palabra: de --> línea: 3, columna: 12

Pareja 3
Palabra: texto --> línea: 3, columna: 15
Palabra: texto --> línea: 3, columna: 21

Pareja 4
Palabra: jaume --> línea: 4, columna: 31
Palabra: jaume --> línea: 5, columna: 1

Pareja 5
Palabra: jaume --> línea: 5, columna: 1
Palabra: jaume --> línea: 5, columna: 7
```

-En el caso que queramos llevar a término la codificación del fichero, introduciremos el número 7. El programa nos indicará cual es la semilla de codificación actual. Nosotros pusimos la semilla por defecto a 5, pero si le introducimos otra semilla, el fichero se codificará en función de la semilla que le hayamos introducido. Para este ejemplo hemos puesto la semilla 8:

```
La semilla actual es: 5
INTRODUCIR NUEVA SEMILLA DE GENERACIÓN ALEATORIA: 8
```

El resultado de esta codificación en un fichero llamado "jj.txt.cod.txt" es la siguiente:

```
jj.txt.cod.txt
lsoe pyc fyigew e xshswe iwxe iw oe tvegxmge jmgeo hi tvskvepegmsg hi nseg c cs
qyiwxvs xvefens gsqwmwxi iq egeomdev yq jmglivs
jmglivs hi hi xibxs xibxs iwxe tvegxmge iw pyc
mptsvxegxi e qmzio hi jmglivs neypi
neypi neypi neypif
```

-Para llevar a cabo el proceso inverso de decodificar un fichero previamente codificado, introduciremos el número 8 en el menú. Como en el caso de la codificación, se nos indicará la semilla actual, que en este caso era 8. Si introducimos el el fichero codificado "jj.txt.cod.txt" como fichero el cual queremos analizar y establecemos la semilla de decodificación 8, el resultado de su ejecución será el contenido del fichero original "jj.txt". Además se mostrará la conversión de cada carácter al nuevo carácter codificado, para que el usuario pueda ver la conversión con facilidad:

```
La semilla actual es: 8
INTRODUCIR NUEVA SEMILLA DE DECODIFICACIÓN: 8
```

-Por último, si queremos salir del menú, introducimos el numero 0 y se visualizará el siguiente mensaje:

```
OPCIONES DE ANÁLISIS
(1) MOSTRAR LA LETRA MÁS REPETIDA Y SU NUMERO DE APARICIONES.
(2) MUESTRA EL NUMERO DE APARICIONES DE CADA CARACTER.
(3) MUESTRA LA PALABRA MÁS REPETIDA Y SU NÚMERO DE APARICIONES.
(4) BUSCA UNA PALABRA.
(5) BUSCA UN TEXTO.
(6) BUSCA LAS PALABRAS REPETIDAS.
(7) CODIFICA FICHERO.
(8) ESTABLECE LA SEMILLA DE DECODIFICACIÓN PARA DECODIFICAR EL FICHERO.
(0) SALIR.

ESCOGER OPCIÓN --> 0
Gracias por utilizar nuestro procesador de textos. Esperamos que tengas un buen día.
BUILD SUCCESSFUL (total time: 5 minutes 33 seconds)
```

4. Conclusión

En conclusión, creemos que esta práctica es muy útil para unificar en cierta manera todos los conocimientos adquiridos este cuatrimestre, tanto desde un simple system.out.print hasta un método de objeto lectura de la clase lineaFicherosIn. Ambos miembros del grupo

hemos tenido buenas sensaciones en la elaboración de este proyecto y consideramos que nuestro aprendizaje se ha visto reflejado. Los métodos más laboriosos de esta práctica han sido el de buscar texto (por el posible calibre del texto a ser buscado) y buscar palabra, ambos métodos siendo complejos en su bucle de tratamiento. A pesar de esto, estamos contentos con los resultados y esperamos con ansias poder volver a repetir algún proyecto similar a este.